

# Centralized Log Management — Simulation Report

## 1. Purpose and scope

- **Objective.** Build a centralised setup with Elastic agents and Grafana, explore an alternative shipper (Vector) and analyser (OpenObserve), and compare them against the criteria in the R&D document.
- **Simulated environment.** The workload is modelled on CBA's publicly described business: top-ups, e-vouchers and bill payments through partner integrations.
- **Synthetic data only.** All phone numbers, bill references, voucher codes and identifiers were randomly generated and belong to no real person.

## 2. Environment

Three Ubuntu Server 24.04 LTS virtual machines ran on VirtualBox on a Windows laptop with 8 GB of RAM.

| VM        | Role                           | Host-only IP  | RAM    | vCPU | Software                                            |
|-----------|--------------------------------|---------------|--------|------|-----------------------------------------------------|
| logserver | Central log server             | 192.168.56.10 | 3 GB   | 2    | Elasticsearch 9.5, Grafana, OpenObserve 1.0.4 (OSS) |
| app1      | Simulated top-up service       | 192.168.56.11 | 768 MB | 1    | Simulator, Filebeat 9.5.5, Vector 0.59.0            |
| app2      | Simulated bill-payment service | 192.168.56.12 | 768 MB | 1    | Simulator, Filebeat 9.5.5, Vector 0.59.0            |

- **Build.** One base VM was installed, then cloned as linked clones; each clone received a new hostname, machine ID and SSH host keys.
- **Network.** Each VM has a NAT adapter for package installation and a host-only adapter with a static address for VM-to-VM and browser access.
- **Storage.** logserver's root logical volume was extended by 8 GB with LVM before Elasticsearch was installed.
- **Recovery points.** A VirtualBox snapshot was taken at each milestone.

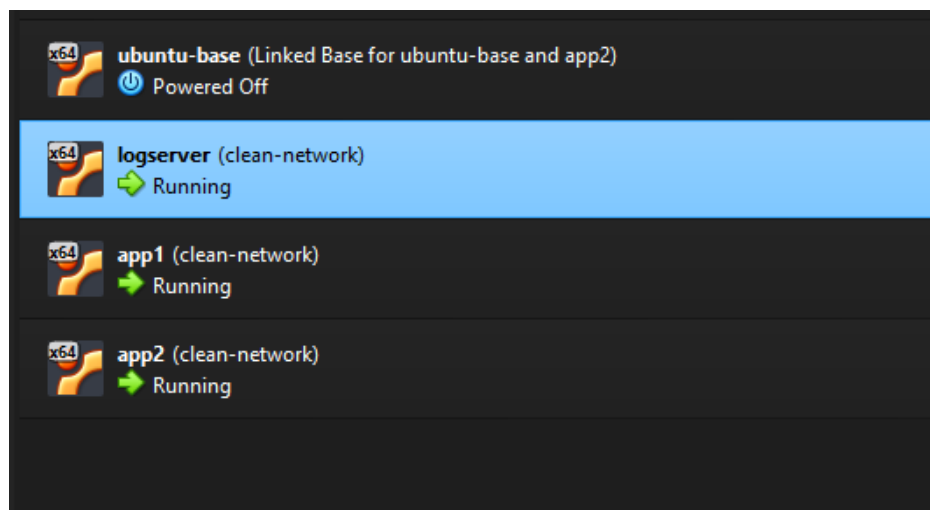


Figure 1: VirtualBox, showing the powered-off linked base and the three running VMs at their *cLean-network* snapshot.

```
(base) PS C:\Program Files\Oracle\VirtualBox> ssh zarmeena@192.168.56.10
zarmeena@192.168.56.10's password:
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-146-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Oct  8 10:04:39 AM UTC 2026

System load:            0.37
Usage of /:             41.3% of 11.21GB
Memory usage:          7%
Swap usage:            0%
Processes:             122
Users logged in:       0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe9d:97f1

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '26.04.1 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Thu Oct  8 09:41:56 2026 from 192.168.56.1
zarmeena@logserver:~$ |
```

Figure 2: SSH from the Windows host to logserver over the host-only network.

```
zarmeena@app1:~$ sudo cat /etc/netplan/50-cloud-init.yaml
[sudo] password for zarmeena:
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false
      addresses:
        - 192.168.56.11/24
```

Figure 3: Static host-only address on app1 (netplan); enp0s3 keeps DHCP for internet access.

```

zarmeena@logserver:~$ ping -c 2 app1
PING app1 (192.168.56.11) 56(84) bytes of data.
64 bytes from app1 (192.168.56.11): icmp_seq=1 ttl=64 time=0.265 ms
64 bytes from app1 (192.168.56.11): icmp_seq=2 ttl=64 time=0.222 ms

--- app1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1030ms
rtt min/avg/max/mdev = 0.222/0.243/0.265/0.021 ms
zarmeena@logserver:~$ ping -c 2 app2
PING app2 (192.168.56.12) 56(84) bytes of data.
64 bytes from app2 (192.168.56.12): icmp_seq=1 ttl=64 time=0.234 ms
64 bytes from app2 (192.168.56.12): icmp_seq=2 ttl=64 time=0.199 ms

--- app2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1006ms
rtt min/avg/max/mdev = 0.199/0.216/0.234/0.017 ms
zarmeena@logserver:~$

```

Figure 4: logserver reaching app1 and app2 by name.

```

(base) PS C:\Program Files\Oracle\VirtualBox> .\VBoxManage.exe snapshot logserver list
Name: clean-network (UUID: e1d8d7b3-8bd1-4e26-9989-587a897f1946) *
(base) PS C:\Program Files\Oracle\VirtualBox>

```

```

(base) PS C:\Program Files\Oracle\VirtualBox> .\VBoxManage.exe snapshot app1 list
Name: clean-network (UUID: 139c82a0-88fd-4dbf-86cb-420c45355cd4) *
(base) PS C:\Program Files\Oracle\VirtualBox> .\VBoxManage.exe snapshot app2 list
Name: clean-network (UUID: 968f23aa-8ae2-47f8-905e-a6ed58400f25) *

```

Figure 5: cLean-network snapshots for logserver (top), app1 and app2 (bottom).

```

zarmeena@logserver:~$ df -h /
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv 12G    4.7G    6.0G   44% /

zarmeena@logserver:~$ df -h /
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv 20G    4.7G    14G   26% /

```

Figure 6: logserver root filesystem before (12 GB) and after (20 GB) the LVM extension.

### 3. Simulated workload

A Python simulator on each app server writes transaction logs as JSON lines, at about 2 transactions per second per service.

| Service         | VM   |
|-----------------|------|
| topup-service   | app1 |
| billpay-service | app2 |

The simulator deliberately reproduces the causes listed in the R&D document, so that each fix could be measured:

- DEBUG logging enabled, writing full partner request and response payloads.
- Sensitive values in plain text: phone numbers, bill references and e-voucher codes.
- A health-check line every 2 seconds.
- An outage switch that makes one partner time out on 85% of calls, with up to three retries per transaction.
- No log rotation at the start.

```

zarneena@app1:~$ systemctl status cba-sim --no-pager
● cba-sim.service - CBA transaction log simulator (synthetic data)
   Loaded: loaded (/etc/systemd/system/cba-sim.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-10-08 10:13:35 UTC; 10min ago
     Main PID: 1175 (python3)
        Tasks: 1 (limit: 753)
      Memory: 7.0M (peak: 7.2M)
         CPU: 747ms
       CGroup: /system.slice/cba-sim.service
               └─1175 /usr/bin/python3 /opt/cba-sim/simulator.py

Oct 08 10:13:35 app1 systemd[1]: Started cba-sim.service - CBA transaction log simulator (synthetic data).

zarneena@app2:~$ systemctl status cba-sim --no-pager
● cba-sim.service - CBA transaction log simulator (synthetic data)
   Loaded: loaded (/etc/systemd/system/cba-sim.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-10-08 10:17:02 UTC; 7min ago
     Main PID: 1255 (python3)
        Tasks: 1 (limit: 753)
      Memory: 6.6M (peak: 6.8M)
         CPU: 543ms
       CGroup: /system.slice/cba-sim.service
               └─1255 /usr/bin/python3 /opt/cba-sim/simulator.py

Oct 08 10:17:02 app2 systemd[1]: Started cba-sim.service - CBA transaction log simulator (synthetic data).
zarneena@app2:~$

```

Figure 7: The simulator running as a systemd service on app1 (top) and app2 (bottom).

```

{
  "timestamp": "2026-10-08T10:25:03.673Z",
  "level": "DEBUG",
  "service": "topup-service",
  "host": "app1",
  "event": "partner_response",
  "attempt": 1,
  "latency_ms": 197,
  "transaction_id": "TXN-E077CB69E346",
  "agent_id": "AGT-0119",
  "partner": "Zong",
  "payload": {
    "code": "00",
    "message": "Success",
    "partner_ref": "73F86BBDE5"
  }
}
{
  "timestamp": "2026-10-08T10:25:03.673Z",
  "level": "INFO",
  "service": "topup-service",
  "host": "app1",
  "event": "transaction_completed",
  "status": "success",
  "attempts": 1,
  "total_ms": 197,
  "transaction_id": "TXN-E077CB69E346",
  "agent_id": "AGT-0119",
  "partner": "Zong"
}
zarneena@app1:~$

```

```

{
  "timestamp": "2026-10-08T10:25:42.918Z",
  "level": "DEBUG",
  "service": "billpay-service",
  "host": "app2",
  "event": "partner_response",
  "attempt": 2,
  "latency_ms": 115,
  "transaction_id": "TXN-3B15FD14F0AE",
  "agent_id": "AGT-0007",
  "partner": "LESCO",
  "payload": {
    "code": "00",
    "message": "Success",
    "partner_ref": "1BBBCA2CDD"
  }
}
{
  "timestamp": "2026-10-08T10:25:42.918Z",
  "level": "INFO",
  "service": "billpay-service",
  "host": "app2",
  "event": "transaction_completed",
  "status": "success",
  "attempts": 2,
  "total_ms": 30115,
  "transaction_id": "TXN-3B15FD14F0AE",
  "agent_id": "AGT-0007",
  "partner": "LESCO"
}
zarmeena@app2:~$

```

Figure 8: Sample records from app1 (top) and app2 (bottom): a DEBUG line carrying the full partner payload, followed by the INFO completion line.

```

zarmeena@app1:~$ grep voucher_issued /var/log/cba-sim/topup-service.log | tail -n 1 | jq .
{
  "timestamp": "2026-10-08T10:26:13.948Z",
  "level": "INFO",
  "service": "topup-service",
  "host": "app1",
  "event": "voucher_issued",
  "voucher_code": "M826-T8AB-F9ER-C8MA",
  "transaction_id": "TXN-9E22B2E933A7",
  "agent_id": "AGT-0009",
  "partner": "Netflix"
}
zarmeena@app1:~$

```

Figure 9: A raw voucher\_issued record on app1, with the voucher code in plain text.

```

zarmeena@app1:~$ jq -c 'select(.partner=="Jazz" and (.event=="partner_timeout" or .event=="transaction_failed")) | {timestamp, transaction_id, event, attempt}' /var/log/cba-sim/topup-service.log | tail -n 9
{"timestamp": "2026-10-08T10:32:57.076Z", "transaction_id": "TXN-87E604FB442E", "event": "partner_timeout", "attempt": 3}
{"timestamp": "2026-10-08T10:32:57.076Z", "transaction_id": "TXN-87E604FB442E", "event": "transaction_failed", "attempt": null}
{"timestamp": "2026-10-08T10:32:58.439Z", "transaction_id": "TXN-175B7907617A", "event": "partner_timeout", "attempt": 1}
{"timestamp": "2026-10-08T10:33:00.924Z", "transaction_id": "TXN-D2F481875A50", "event": "partner_timeout", "attempt": 1}
{"timestamp": "2026-10-08T10:33:00.924Z", "transaction_id": "TXN-D2F481875A50", "event": "partner_timeout", "attempt": 2}
{"timestamp": "2026-10-08T10:33:00.924Z", "transaction_id": "TXN-D2F481875A50", "event": "partner_timeout", "attempt": 3}
{"timestamp": "2026-10-08T10:33:00.924Z", "transaction_id": "TXN-D2F481875A50", "event": "transaction_failed", "attempt": null}
{"timestamp": "2026-10-08T10:33:04.114Z", "transaction_id": "TXN-BE722767D85F", "event": "partner_timeout", "attempt": 1}
{"timestamp": "2026-10-08T10:33:04.114Z", "transaction_id": "TXN-BE722767D85F", "event": "partner_timeout", "attempt": 2}

```

Figure 10: Retry storm during the simulated Jazz outage: each transaction times out on attempts 1–3 and then fails.

```

zarmeena@app2:~$ jq -r '.event' /var/log/cba-sim/billpay-service.log | sort | uniq -c | sort -rn
1480 request_received
1480 partner_request
1465 transaction_completed
1465 partner_response
288 health_check
88 partner_timeout
15 transaction_failed
zarmeena@app2:~$ jq -r '.level' /var/log/cba-sim/billpay-service.log | sort | uniq -c | sort -rn
3340 INFO
3043 DEBUG
88 WARN
15 ERROR
zarmeena@app2:~$

```

Figure 11: Where the volume comes from on app2: DEBUG lines are 3,043 of 6,486 lines (47%), and health checks add 288 more.

## 4. Part A: Filebeat → Elasticsearch → Grafana

Filebeat on both app servers shipped the simulator logs to a single-node Elasticsearch, and Grafana displayed them on a five-panel dashboard. Every account involved had only the permissions it needed.

### Setup and security choices

- **Elasticsearch:** installed from Elastic's APT repository with a 1 GB memory cap. Its automatic security configuration (password authentication and TLS) was kept.
- **Filebeat account:** filebeat\_app, with a role allowing it only to write logs and manage its own index settings. Its password is held in Filebeat's keystore, not the config file.
- **Certificate checking:** Filebeat verifies Elasticsearch's certificate on every connection.
- **Field namespacing:** simulator fields were placed under cba.\*, because fields named host, service and event would otherwise collide with fields Filebeat adds.
- **Event time:** a timestamp processor preserves the time each transaction happened rather than the time it arrived.
- **Grafana account:** grafana\_reader, with read-only access to the log indices.
- **Grafana as code:** the data source and dashboard are provisioned from files, with the password and certificate read from protected files.

**Dashboard panels:** transactions per minute by service; partner timeouts and failures; log volume by level; log volume by event type; transaction lookup by ID.

### Observations before hygiene fixes

- DEBUG lines made up about 46% of all lines: roughly 28,700 of 62,000 in one hour.
- Health checks added about 2,900 lines per hour.
- Baseline partner timeouts were about 3% of transactions (425 of 14,365 in one hour).
- A simulated Jazz outage appeared clearly on the partner panel within minutes.
- A single transaction's journey could be retrieved centrally by its ID.

```

elastic password.
zarmeena@logserver:~$ sudo curl -s --cacert /etc/elasticsearch/certs/http_ca.crt -u "elastic:$ES_PASS" "https://localhost:9200/_cluster/health?pretty"
{
  "cluster_name" : "elasticsearch",
  "status" : "green",
  "timed_out" : false,
  "number_of_nodes" : 1,
  "number_of_data_nodes" : 1,
  "active_primary_shards" : 3,
  "active_shards" : 3,
  "relocating_shards" : 0,
  "initializing_shards" : 0,
  "unassigned_shards" : 0,
  "unassigned_primary_shards" : 0,
  "delayed_unassigned_shards" : 0,
  "number_of_pending_tasks" : 0,
  "number_of_in_flight_fetch" : 0,
  "task_max_waiting_in_queue_millis" : 0,
  "active_shards_percent_as_number" : 100.0
}

```

Figure 12: Elasticsearch cluster health: green, single node.

```

zarmeena@app1:~$ sudo filebeat test output
elasticsearch: https://logserver:9200...
  parse url... OK
  connection...
    parse host... OK
    dns lookup... OK
    addresses: 192.168.56.10
    dial up... OK
  TLS...
    security: server's certificate chain verification is enabled
    handshake... OK
    TLS version: TLSv1.3
    dial up... OK
  talk to server... OK
  version: 9.5.5
zarmeena@app1:~$ |

```

```

zarmeena@app2:~$ sudo filebeat test output
elasticsearch: https://logserver:9200...
  parse url... OK
  connection...
    parse host... OK
    dns lookup... OK
    addresses: 192.168.56.10
    dial up... OK
  TLS...
    security: server's certificate chain verification is enabled
    handshake... OK
    TLS version: TLSv1.3
    dial up... OK
  talk to server... OK
  version: 9.5.5

```

Figure 13: Filebeat output test on app1 (top) and app2 (bottom): certificate chain verified, TLS 1.3, server reached.

```

zarmeena@logserver:~$ sudo curl -s --cacert /etc/elasticsearch/certs/http_ca.crt -u "elastic:$ES_PASS" "https://localhost:9200/filebeat-*/_count?pretty"
{
  "count" : 86063,
  "_shards" : {
    "total" : 1,
    "successful" : 1,
    "skipped" : 0,
    "failed" : 0
  }
}

```

Figure 14: Log lines arriving in Elasticsearch (86,063 at the time of capture).

```

zarmeena@logserver:~$ sudo curl -s --cacert /etc/elasticsearch/certs/http_ca.crt -u "elastic:$ES_PASS" \
-H 'Content-Type: application/json' "https://localhost:9200/filebeat-*/_search?size=0&pretty" -d '{
  "aggs": { "by_service": { "terms": { "field": "cba.service" } } }
}'
{
  "took" : 53,
  "timed_out" : false,
  "_shards" : {
    "total" : 1,
    "successful" : 1,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : {
      "value" : 10000,
      "relation" : "gte"
    },
    "max_score" : null,
    "hits" : [ ]
  },
  "aggregations" : {
    "by_service" : {
      "doc_count_error_upper_bound" : 0,
      "sum_other_doc_count" : 0,
      "buckets" : [
        {
          "key" : "topup-service",
          "doc_count" : 45482
        },
        {
          "key" : "billpay-service",
          "doc_count" : 41624
        }
      ]
    }
  }
}

```

Figure 15: Both services reporting centrally: topup-service and billpay-service.

```

zarmeena@logserver:~$ sudo curl -s --cacert /etc/elasticsearch/certs/http_ca.crt -u "elastic:$ES_PASS" \
"https://localhost:9200/filebeat-*/_search?q=cba.transaction_id:TXN-B2CBD4C64248&pretty" | grep -E '"event"|"transaction_id"|"partner"|"
status"'
    "transaction_id" : "TXN-B2CBD4C64248",
    "partner" : "Zong",
    "event" : "request_received",
    "event" : "partner_request",
    "transaction_id" : "TXN-B2CBD4C64248",
    "partner" : "Zong",
    "partner" : "Zong",
    "event" : "partner_response",
    "transaction_id" : "TXN-B2CBD4C64248",
    "event" : "transaction_completed",
    "status" : "success",
    "partner" : "Zong",
    "transaction_id" : "TXN-B2CBD4C64248"

```

Figure 16: Central lookup of transaction TXN-B2CBD4C64248, from request\_received to transaction\_completed.

```

zarmeena@logserver:~$ sudo curl -s --cacert /etc/elasticsearch/certs/http_ca.crt -u "elastic:$ES_PASS" "https://localhost:9200/_cat/index
s/.ds-filebeat-*?v&h=index,docs.count,store.size&expand_wildcards=all"
index              docs.count store.size
.ds-filebeat-9.5.5-2026.10.08-000001  90261      23.1mb

```

Figure 17: Filebeat index size before the hygiene fixes: 90,261 records, 23.1 MB.

✓ Elasticsearch data source is healthy.

Next, you can start to visualize data by [building a dashboard from scratch](#) or by querying data in the [Explore view](#).

Figure 18: Grafana's provisioned Elasticsearch data source passing its health check with the read-only account.

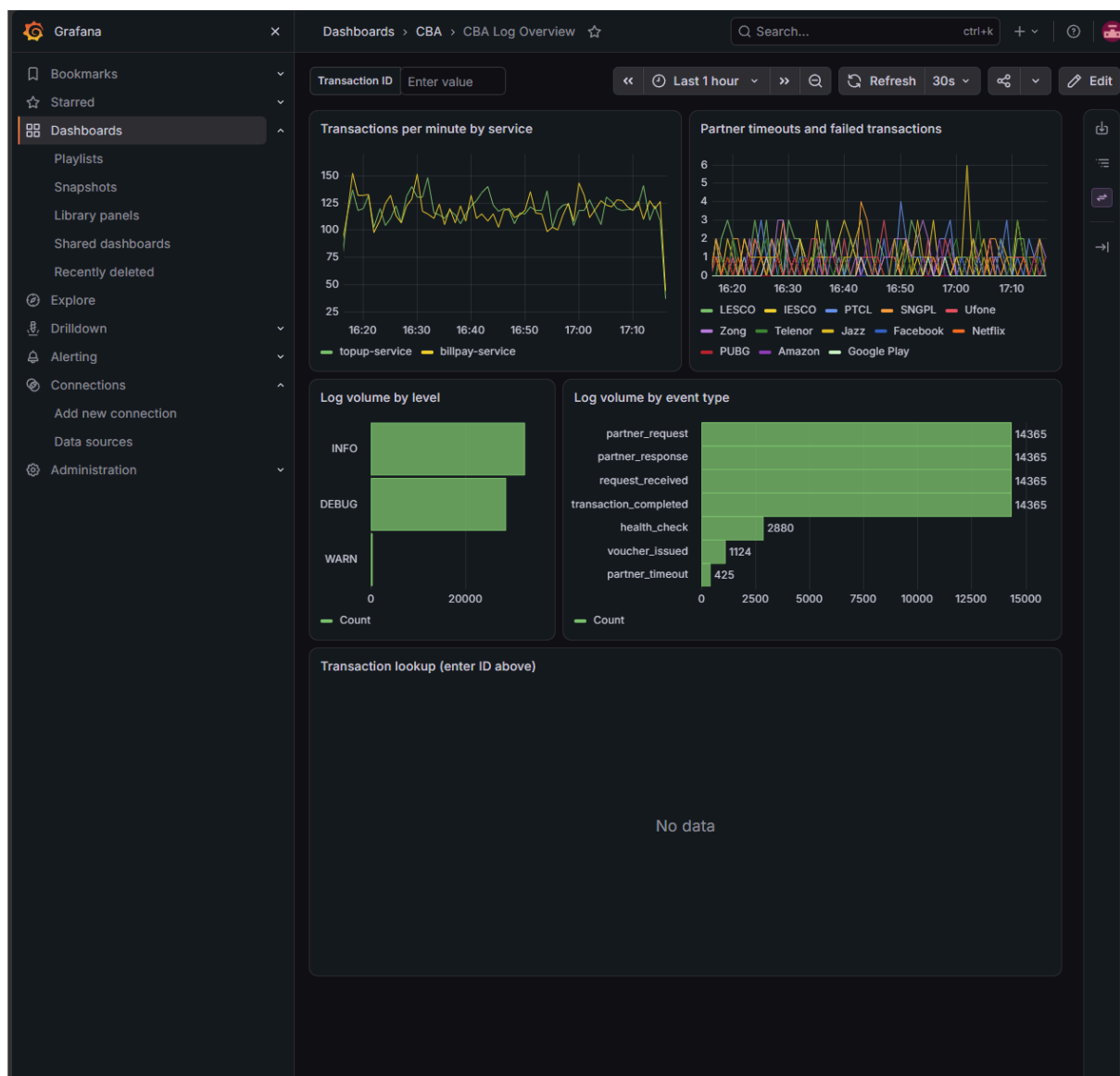


Figure 19: Grafana dashboard before the hygiene fixes: DEBUG lines and health checks make up a large share of volume.

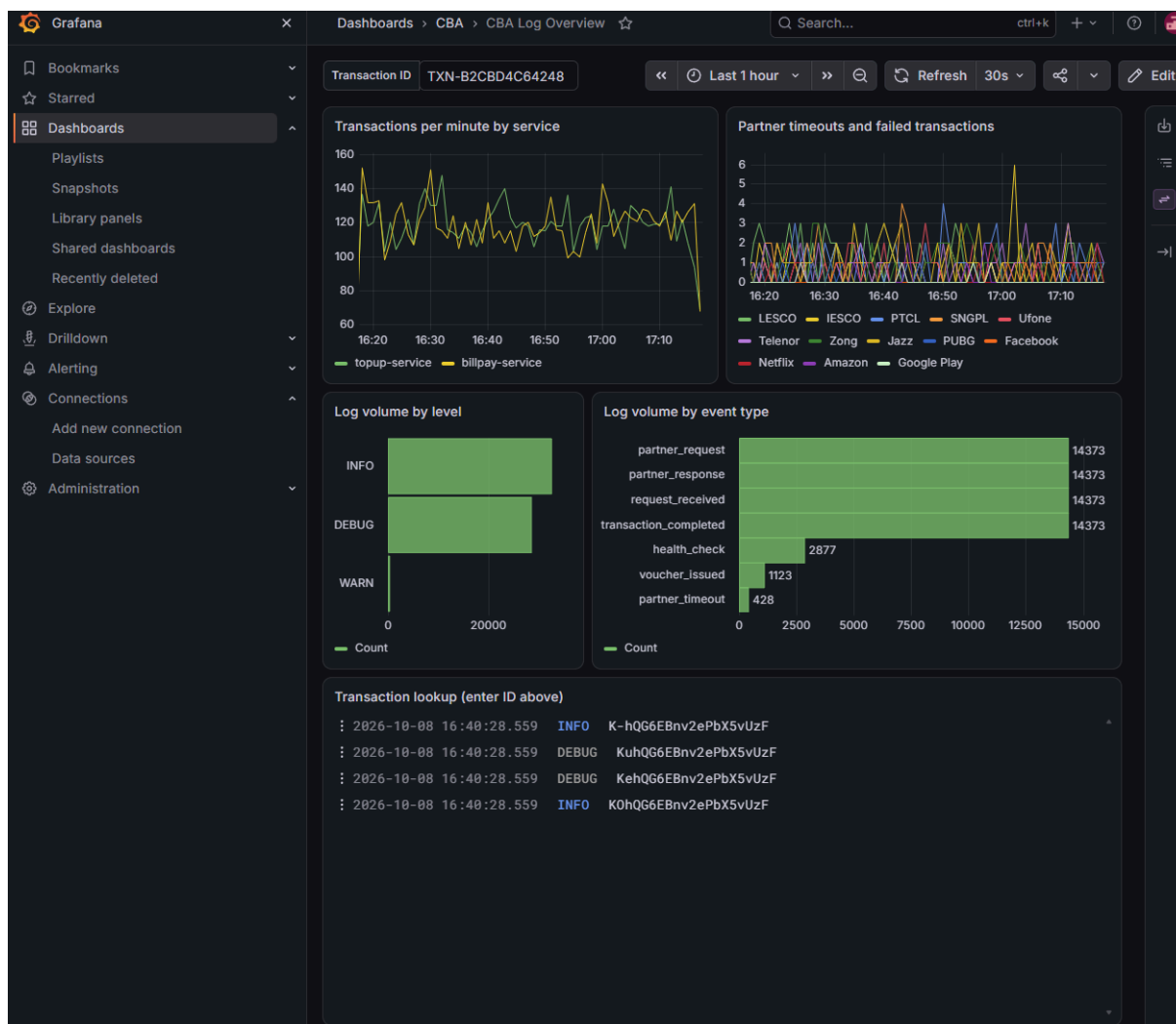


Figure 20: Grafana dashboard with a transaction ID entered in the lookup panel.

## 5. Log hygiene and retention

Applying the R&D document's solution levers cut local log growth on app1 by 65% and central intake by 53%, with no change to the backend.

| Lever                   | Change applied                                                                                                                       |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Reduce at source        | DEBUG logging switched off                                                                                                           |
| Filter in pipeline      | Filebeat drops health-check events before sending                                                                                    |
| Clean up source servers | logrotate: daily or at 100 MB, keep 3, compressed (delaycompress so Filebeat finishes reading); journald capped at 200 MB on all VMs |
| Retention               | Elasticsearch lifecycle policy: new index daily, data deleted after 7 days                                                           |

| Measure                                      | Before  | After  | Change |
|----------------------------------------------|---------|--------|--------|
| app1 log growth (bytes in one minute)        | 181,151 | 63,529 | -65%   |
| Central log lines, both services (5 minutes) | 5,219   | 2,458  | -53%   |

- **Method.** Both measurements compare fixed past time windows before and after the change (12:10–12:15 and 12:35–12:40 UTC), read from the local files and from Elasticsearch.
- **Bytes fell more than lines.** DEBUG lines carried full payloads and were the largest lines, so removing them saved disproportionately more disk.
- **Prediction matched.** Before the change, DEBUG and health checks made up about 51% of lines; the measured central drop was 53%.

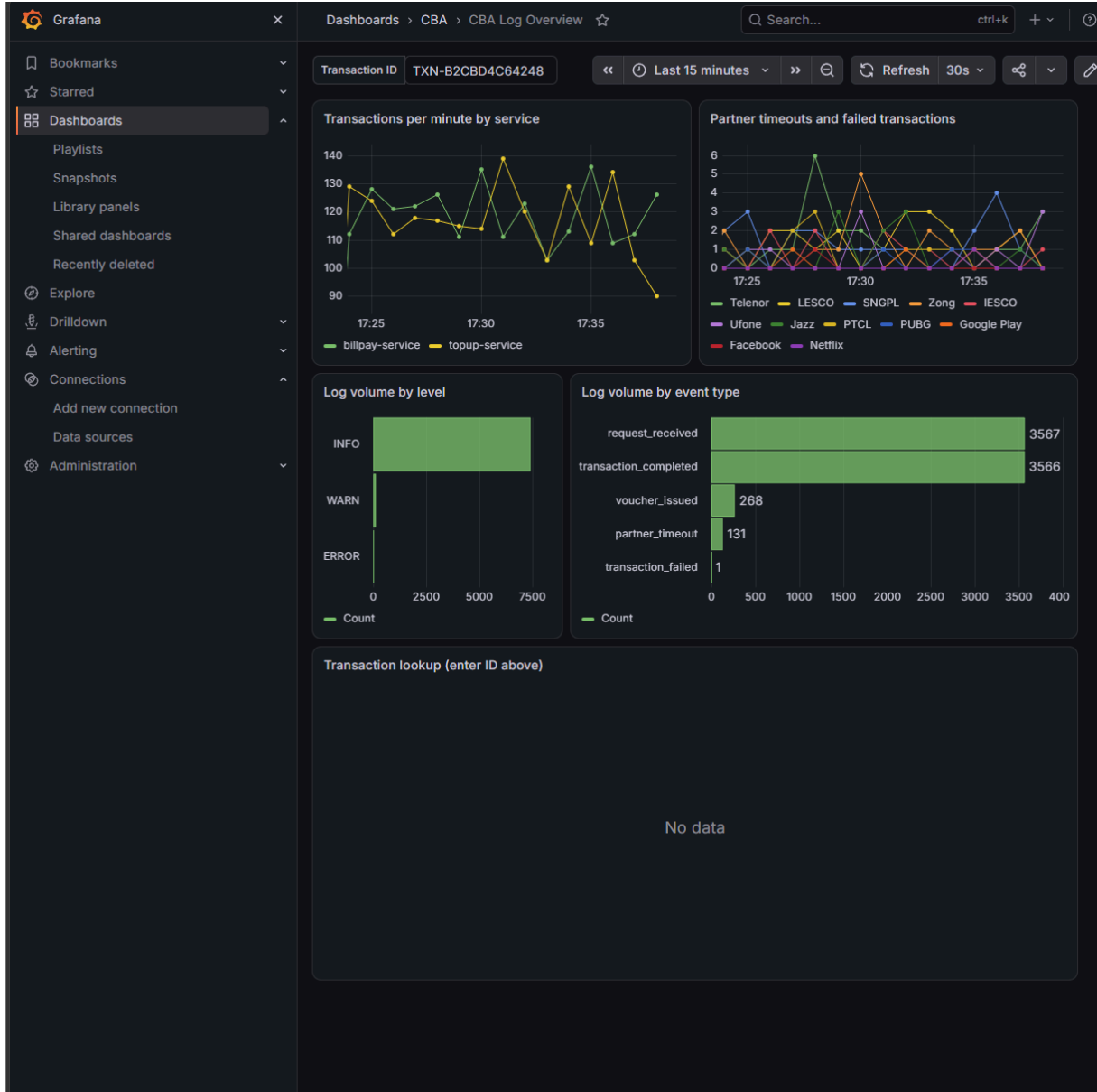


Figure 21: Grafana dashboard after the hygiene fixes (last 15 minutes): DEBUG and health-check events no longer appear.

## 6. Part B: Vector → OpenObserve

Vector on both app servers masked sensitive values before they left the server, and OpenObserve received only masked data. Every voucher code reached central storage as [REDACTED].

### Vector pipeline (per app server)

1. **Parse:** each JSON line becomes structured fields, and the original event time is kept.
2. **Drop noise:** health-check events are discarded before sending.
3. **Mask at source:**

- Phone number shown as 0313\*\*\*\*\*35, plus a salted SHA-256 fingerprint (msisdn\_hash).
  - Bill reference reduced to its last 4 digits.
  - Voucher code replaced with [REDACTED].
  - The same rules apply inside DEBUG payloads if DEBUG is ever re-enabled.
4. **Deliver:** sent to OpenObserve with a 256 MB disk buffer and end-to-end acknowledgements, so a line counts as delivered only once it is stored. Credentials come from Vector's secrets file.

## Verification

- **Voucher masking:** in one hour, 1,040 of 1,040 voucher records were stored as [REDACTED].
- **Phone numbers:** stored only in masked form, for example 0313\*\*\*\*\*35.
- **Salt:** recomputing one record's fingerprint from the original number and the secret salt matched the stored value (75bb3e41d0b60148...); without the salt it would not.
- **Support lookup:** searching by fingerprint returned the customer's transaction, so phone-number lookup works without the number being stored centrally.

**OpenObserve setup:** OpenObserve 1.0.4 (open-source edition) runs as a single binary under systemd with a dedicated user. Its dashboard was generated by script from an exported template and imported through the API. A masking-coverage panel acts as a standing compliance check: any value other than [REDACTED] would indicate a leak.

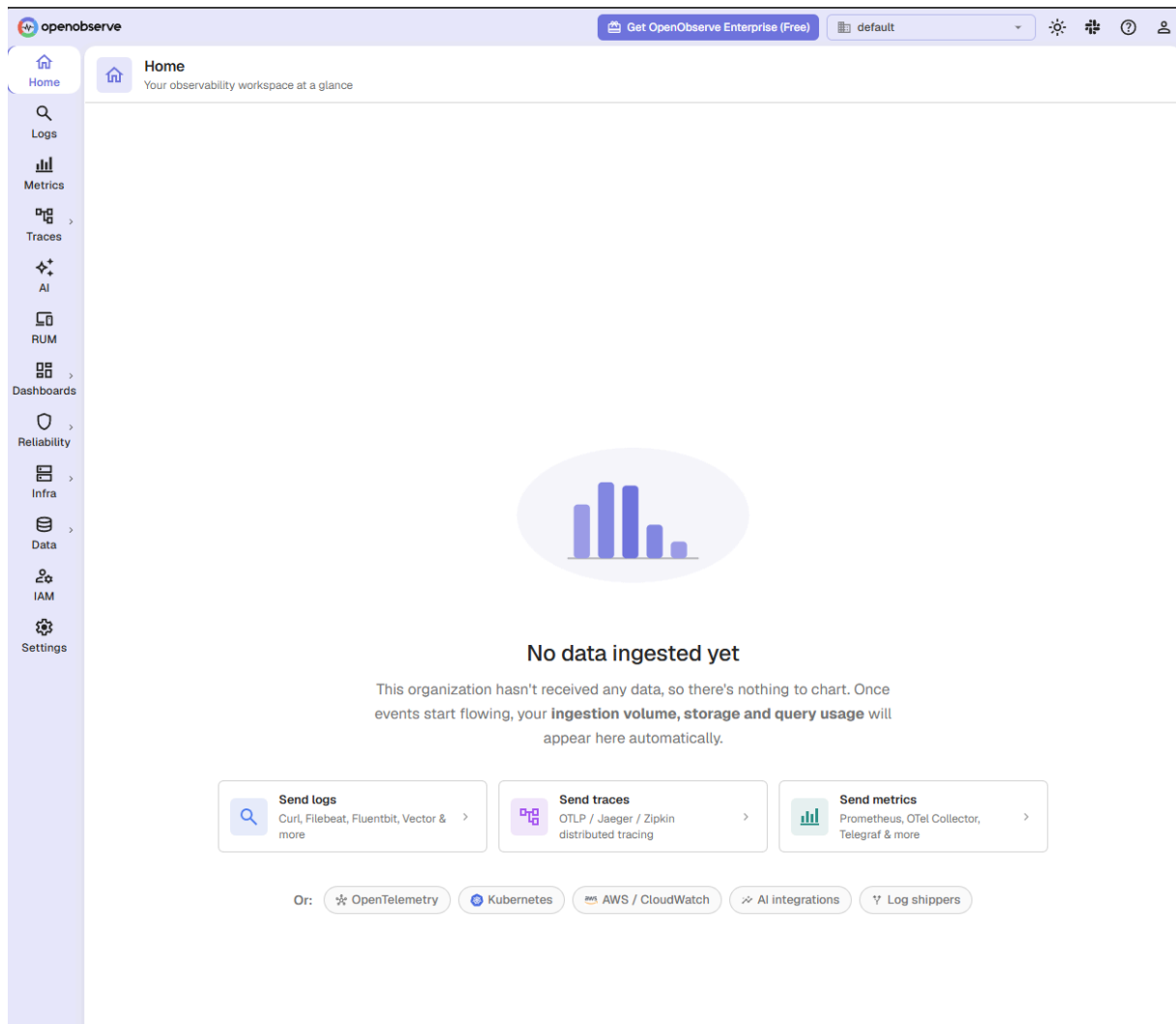


Figure 22: OpenObserve running on logserver after installation, before any data was ingested.

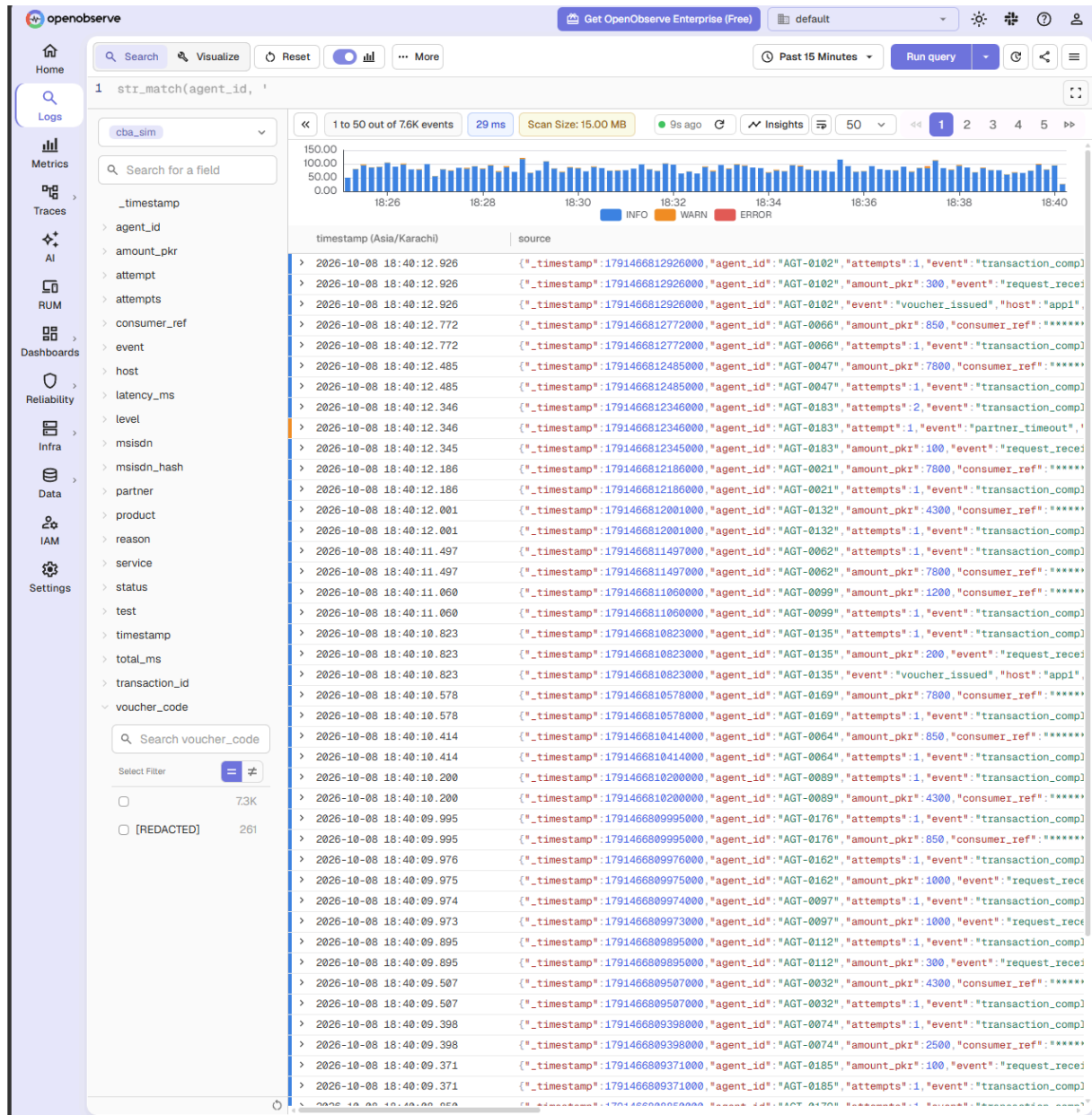


Figure 23: The `cba_sim` stream in OpenObserve: the `voucher_code` field holds only `[REDACTED]` values, and bill references arrive masked.

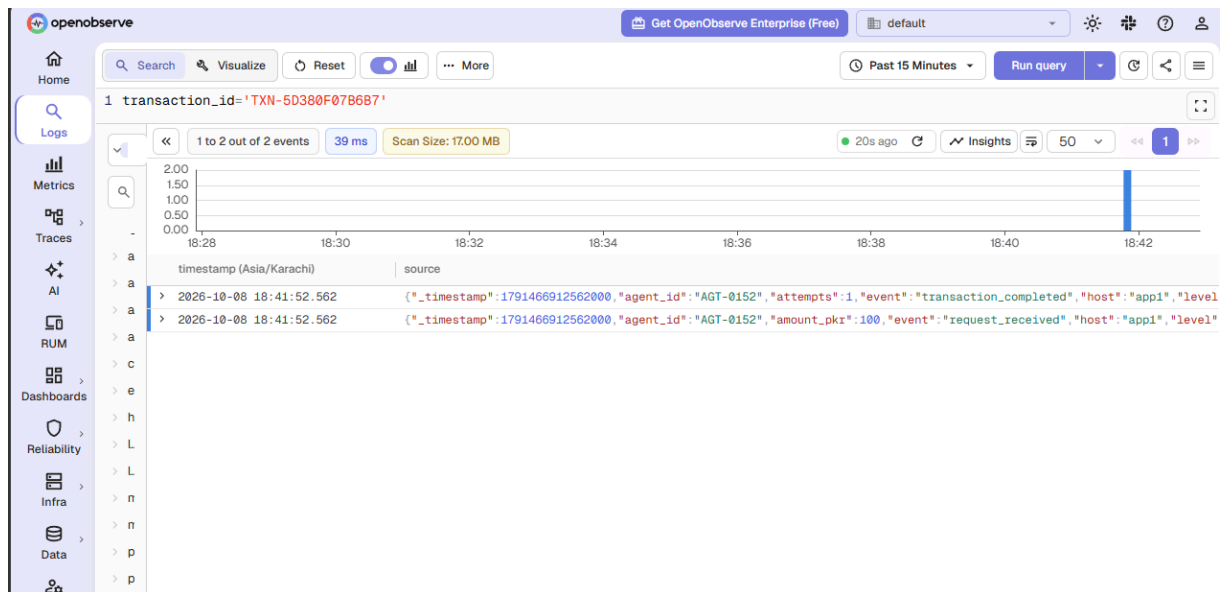


Figure 24: Lookup of transaction TXN-5D380F07B6B7 in OpenObserve: 2 events found in 39 ms, scanning 17 MB.



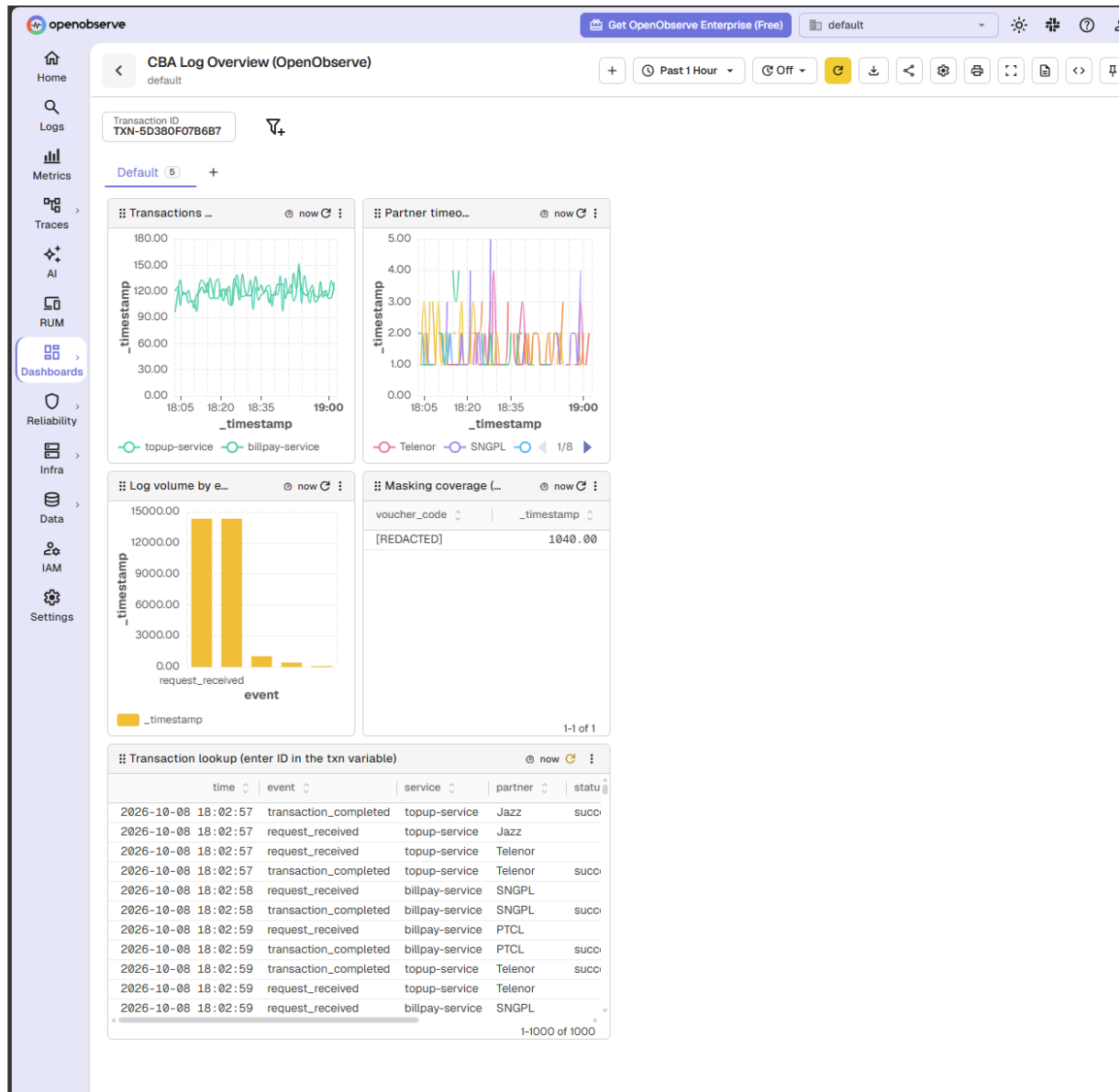


Figure 25: The imported OpenObserve dashboard (top) and the same dashboard with a transaction ID entered (bottom). The masking-coverage panel shows every voucher code in the hour stored as [REDACTED].

## 7. Part C: Elasticsearch vs OpenObserve

On identical data, OpenObserve used 5–7× less disk and 3.6× less memory; Elasticsearch found new transactions about 10× faster and had stronger security and failure behaviour by default.

### Method

- **Same data:** 1,000,000 log lines spread over 24 hours, generated with a fixed random seed so the dataset can be reproduced exactly.
- **Same shipper:** Vector with identical parsing and masking sent to both backends, so only the backend differed.
- **One backend at a time** on the same VM.
- **Settled storage:** measured after OpenObserve's compaction and after an Elasticsearch force-merge to one segment.
- **Lookups:** 8 transaction IDs aged about 1–23 hours, 3 runs each, over a 26-hour window. Server-side processing times are reported; caching was disabled in Elasticsearch.

## Results

| Measure                        | Elasticsearch (standard)                   | Elasticsearch (logsdb) | OpenObserve (OSS)                                                  |
|--------------------------------|--------------------------------------------|------------------------|--------------------------------------------------------------------|
| Records stored                 | 1,000,000                                  | 1,000,000              | 1,000,000, only after raising the age limit (default kept 207,157) |
| Disk used, settled             | 151 MB                                     | 105 MB                 | 20.3 MB                                                            |
| Compression vs 293 MB received | 1.9×                                       | 2.8×                   | 14.5×                                                              |
| RAM with data loaded           | 1,514 MB                                   | Same node              | 424 MB                                                             |
| Lookup, new transaction ID     | 2–5 ms                                     | 2–5 ms                 | 35–37 ms (scans all 293 MB)                                        |
| Lookup, repeated               | 2–5 ms (cache off)                         | 2–5 ms                 | 6–8 ms (results cache)                                             |
| Encryption in transit, default | TLS on                                     | TLS on                 | Plain HTTP                                                         |
| Access control, free edition   | Roles (write-only, read-only, append-only) | Same                   | All users have full access                                         |
| Rejected data                  | Explicit error, logged by Vector           | Same                   | Silent (HTTP 200, data discarded)                                  |

- **Lookups scale differently.** Elasticsearch's index locates a record directly, so its time stays flat as data grows. OpenObserve scans the whole time window, so its time grows with data volume and window width.
- **logsdb on the free licence** cut Elasticsearch's disk use by 30% with no lookup slowdown. It ran with source mode STORED; the larger savings of synthetic source mode were not tested.
- **Timing comparability.** Wall-clock times are not compared, because only Elasticsearch used HTTPS.

## 8. Conclusions and recommendations

The simulation supports the R&D document's shortlist: Vector for collection and masking, Elasticsearch in logsdb mode for search, and Grafana for dashboards, with retention and source-side hygiene controlling storage.

### Conclusions

- **Hygiene is the cheapest lever.** Turning off DEBUG and dropping health checks cut log growth by 65% before any backend choice.
- **Masking at source is practical and verifiable.** Vector removed voucher codes and phone numbers before they left the server, and the salted fingerprint preserved phone-number lookups.
- **Elasticsearch fits CBA's criteria better.** It offers flat, fast transaction lookups, encryption and least-privilege accounts by default, and explicit rather than silent failures.
- **OpenObserve is far cheaper to store, but its open-source edition is not suitable as tested** for a compliance-sensitive company: no role-based access, plain HTTP by default and silent data discards.